MODULE 3

DAY 1 · MODULE 3 OF 8

# Establishing Governance and Writing a Clarified Specification

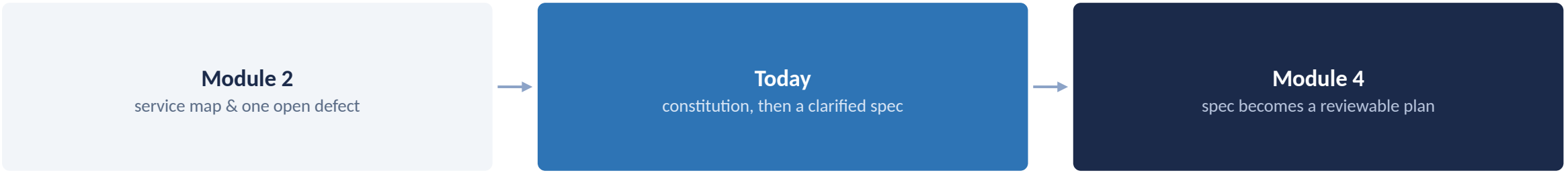
*Set the standard everything else gets checked against, then write against it.*

# What You'll Learn

1. Encode project or per-service governance in a constitution
2. Write a spec from both a feature request and a defect, using EARS (Easy Approach to Requirements Syntax)
3. Run a clarify pass to surface ambiguity before planning
4. Write testable, unambiguous acceptance criteria
5. Map the SDD lifecycle onto your team's existing SDLC

WHY THIS MATTERS

# *You now understand the system, today you set the standard it gets checked against.*




*Everything you write today is what the rest of the lifecycle gets measured against*

PART 1

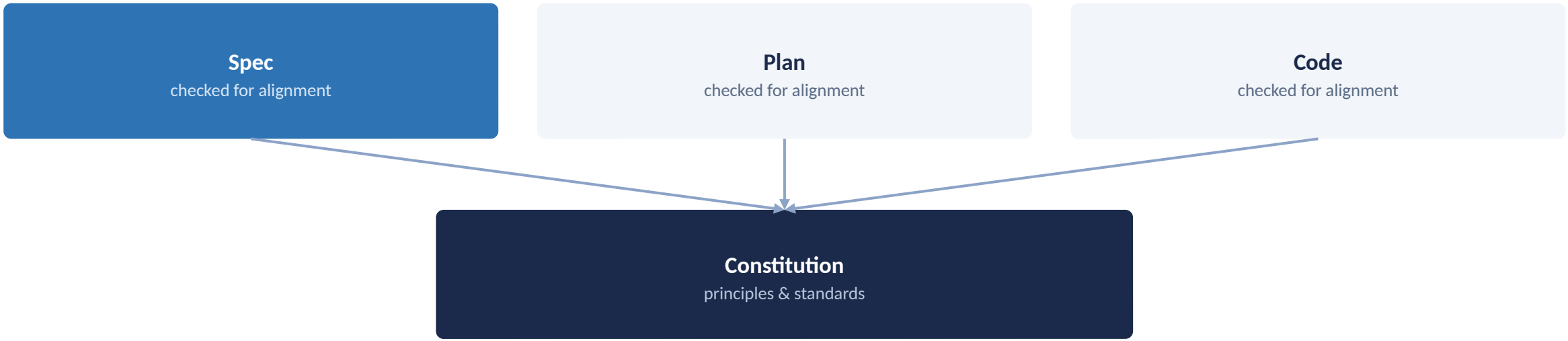
# Governance First

*Why the constitution comes before the spec, not after*

MODULE 3

GOVERNANCE FIRST

# Every Later Artifact Gets Checked Against the Constitution




*Every later artifact is checked against the constitution, not against opinion*

MODULE 3

REAL-WORLD CONTEXT

# A Constitution Encodes Four Kinds of Standing Decisions

**Principles**
the standard "good" gets judged against

**Code quality standards**
style, structure, maintainability

**Testing requirements**
what must be covered before merge

**Architectural constraints**
boundaries a plan can't cross

*Encode your service's existing conventions here, don't invent new ones on the spot*

CHECKPOINT

# Let's Pause and Verify

1 Name two things a constitution should encode, besides principles.

2 Why does every later artifact get checked against the constitution, not against opinion?

PART 2

# Writing the Spec

*EARS-style requirements, two starting points, and a dedicated clarify pass*

# Ambiguity Gets Resolved Before It Reaches a Plan, Not After


*EARS = Easy Approach to Requirements Syntax: "When/if [trigger], the system shall [behavior]", a format that forces testability*

TWO STARTING POINTS

# A Spec Can Start From a Feature Request or a Defect


*same spec format, same rigor, two different starting points*

*Module 2's defect pull-in is exactly what feeds the right-hand path today*

COMMON PITFALLS

# If You Can't Write a Test From It, It's Not Done Yet


*the second one, and only the second one, can fail a test*

*This is the single most common gap a clarify pass catches: criteria nobody can actually check*

CHECKPOINT

# Let's Pause and Verify

1 What's the difference in starting point between a feature-request spec and a defect spec?

2 What makes an acceptance criterion testable rather than vague?

PART 3

# This Isn't a New Process

*Where the SDD lifecycle maps onto the SDLC your team already runs*

MODULE 3

SDLC MAPPING

# The SDD Lifecycle Maps Onto the SDLC You Already Run


*The SDD lifecycle maps onto the SDLC you already run, step for step*

MODULE 3

WHERE THIS FITS

# Both Frameworks Handle Governance and Clarify, at Different Levels of Ceremony

**GitHub Spec Kit**
/speckit.constitution and /speckit.clarify — dedicated commands, generated documents

**OpenSpec**
governance lives in config.yaml, not a generated doc; clarify is distributed across /opsx:explore and /opsx:update

*same governance & clarify concepts, different level of ceremony*

*Neither approach skips these steps, they just live in different places*

CHECKPOINT

# Let's Pause and Verify

1 Which SDD stages map to your team's QA & test planning phase?

2 Which framework keeps governance as a generated document, and which keeps it as a config file?

COMMON PITFALLS

# *A constitution written after the first spec is already too late.*


*the order is the whole point*

*If your team is debating a spec's quality standard mid-review, the constitution came too late*

# Key Takeaways

1. Governance has to exist before code is judged against it, otherwise every review starts from a different, unstated standard.

2. EARS (Easy Approach to Requirements Syntax) keeps requirements testable regardless of framework, the format, not the tool, is what makes a requirement checkable.

3. A spec can start from a feature request or a defect pulled from the tracker, same format, same rigor either way.

4. Clarify early: it's cheap now and expensive once a plan is built on top of ambiguity nobody caught in time.

5. Next: Module 4, From Clarified Spec to Reviewable Plan, turns today's clarified spec into a technical plan your team reviews before any code changes.